Department of Electrical Engineering

Communication Latency in Tensor-Parallel Distributed Large Language Model Inference

by

Penners, K.

MSc Thesis

Thesis committee

Chair:	Prof. A. Alvarado
Member 1:	Prof. N. Calabretta
Member 2:	Assistant Prof. A. Balatsoukas-Stimming
Member 3:	Assistant Prof. Ö. Babur
Advisory Member:	Dr. F. Karinou

Graduation

Program:	Artificial Intelligence & Engineering Systems
Research group:	Signal Processing Systems - ICT Lab
Thesis supervisor:	Prof. A. Alvarado
Date of defense:	11-09-2026
Student ID:	2198525
Study load (ECTS):	45
Track:	High-Tech Systems and Robotics

This thesis is public and Open Access.

This thesis has been realized in accordance with the regulations as stated in the TU/e Code of Scientific Conduct.

Disclaimer: the Department of Electrical Engineering of the Eindhoven University of Technology accepts no responsibility for the contents of MSc theses or practical training reports.

Communication Latency in Tensor-Parallel Distributed Large Language Model Inference

***Abstract*—The rapid scaling of large language models has made distributed serving across multiple accelerators necessary to meet their memory and compute demands. However, distributing inference requires frequent communication between GPUs, introducing overhead that limits model performance. This paper studies how interconnect quality and communication latency affect intra-node tensor-parallel inference using controlled transport configurations and a GPU-side latency injection mechanism. We examine how factors such as model size, request concurrency, and hardware topology affect inference latency and resource efficiency. The results show that optimal serving configurations depend on hardware topology and model architecture, rather than on compute and memory budgets alone. Through controlled latency injection, we demonstrate that decode latency scales linearly with AllReduce communication latency at a rate dependent on model depth. Furthermore, we find that falling back from NVLink to host-side transports degrades scaling efficiency and increases energy costs by up to 90–365% as GPU synchronization stalls leave hardware idle. The findings reveal how the communication layer dictates the performance, energy cost, and operational limits of distributed inference systems, with direct implications for the request capacity and communication budgets that deployments can sustain.**

I. Introduction

Large language models (LLMs) are widespread, powering endless applications including conversational bots, coding assistants, and most recently, autonomous multi-agent systems. Frontier models have grown exponentially in size (Fig. 1), scaling in ways that make it impossible for individual graphics processing units (GPUs) to keep up [1], forcing practitioners to distribute their models across hardware systems. Distribution is now practically a requirement for deployment of these large-scale LLMs.

While training of these models is resource-intensive, requiring an unprecedented number of GPU hours to attain high quality and accuracy, inference is a continuous operational expense driven by user requests [2]. To serve these production applications, models must be deployed on systems that continuously handle thousands of requests under strict latency constraints to maintain user experience. Serving frameworks rely on various parallelism schemes to partition models across multiple GPUs or machines, requiring collective communication operations (e.g., `AllReduce`, which combines intermediate activations across devices) that are typically managed by hardware-specific libraries such as NVIDIA's Collective Communication Library (NCCL). However, this distribution introduces communication overhead as GPUs synchronize during execution. Training workloads typically operate with compute-heavy backward passes with large batch sizes, which can overlap partially with communication latency [3]. Inference,

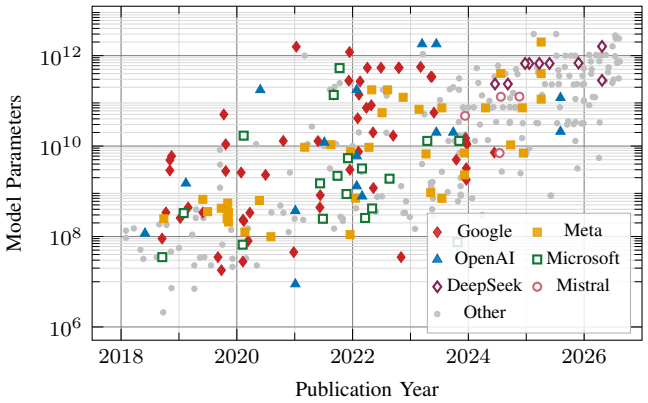


Fig. 1. Parameter counts of LLMs published between 2018 and 2026 across major AI organizations shown on a logarithmic scale (data from [4]).

on the other hand, operates in two phases: the prefill phase, which processes input sequences in parallel, and the decode phase, where output tokens are generated autoregressively, one by one (Fig. 2). The latter requires little computation to overlap with communication operations, leaving execution more sensitive to interconnect latency, which directly impacts user experience, cost of operation, and system design.

The inherent communication overhead in distributed systems has been well-studied in the context of model training, where many studies recognize communication as a primary training bottleneck. Previous work has evaluated factors such as link bandwidth [1], inter-node protocols [5], and compute-vs-communication time breakdowns [6], [7]. For example, in [5], the authors show that the choice of inter-node communication protocol (e.g, GPUDirect Remote Direct Memory Access (RDMA) vs. TCP/IP) significantly impacts training performance. In related work, [8] analyzes Ethernet link latency, finding that the physical layer latency has minimal impact on training time, leaving open whether the same holds for inference.

Although communication characteristics in distributed inference remain under-characterized, existing work by [9] evaluates parallelism methods on serving performance and establishes that communication overhead takes up a larger portion of execution time in inference than during training, particularly due to the high communication frequency of the decode phase. However, the hardware stack that underlies distributed serving—the accelerators that perform matrix multiplications and the physical interconnects that enable data transfer, ranging from direct, high-performance links

like NVLink to standard PCIe buses—shapes the severity of that extra communication cost. How interconnect quality and communication latencies impact phase-specific performance and resource efficiency remains unclear, motivating this work.

Specifically, this paper investigates intra-node distributed LLM inference, focusing on how tensor-parallel execution interacts with the underlying communication layer. We evaluate dense transformer architectures parallelized using tensor parallelism across multi-GPU server topologies. The main contributions of this work include:

- An empirical characterization of tensor parallelism scaling across three dense model architectures (up to 13B parameters) and two GPU generations (A100 and V100), demonstrating phase-specific execution and KV cache saturation behavior.
- A controlled comparison of NCCL transport tiers (NVLink Peer-to-Peer, PCIe Shared Memory, and TCP Sockets) that isolates interconnect quality, quantifying how lower-tier transports degrade scaling and energy efficiency.
- A GPU-side latency injection mechanism and delay model that isolates AllReduce latency, establishing linear decode degradation based on model depth and providing a lower bound on transport fallback overhead using microbenchmarks.
- An evaluation of Service Level Objective (SLO) attainment under scaling and controlled communication latency, providing request capacities and latency tolerances across evaluated system configurations.

The remainder of this paper is organized as follows. Sec. II reviews foundational concepts underlying distributed LLM inference. Sec. III outlines the hardware, models, transport configurations, and the latency injection method used. Sec. IV empirically characterizes distributed inference behavior through single-GPU baselines and tensor-parallel scaling, while Sec. V isolates the effect of transport quality and communication latency on latency, throughput, and energy metrics. Lastly, Sec. VI concludes this work and outlines potential future research.

II. Background

LLMs are typically transformer-based architectures that autoregressively predict output tokens using self-attention over previously generated outputs [10]. Token embeddings are linearly projected into query, key, and value vectors, which are used to compute attention weights through scaled dot-product attention. Standard multi-head attention (MHA) increases the model's representational capacity by running multiple independent attention heads in parallel, each mapping input embeddings into a different learned representational subspace. However, to reduce the computational and memory costs this requires, many modern transformers use grouped query attention (GQA) where multiple query heads share the same key-value projections, balancing representational capacity and memory/compute overhead [11]. The choice of attention mechanism directly impacts an LLM's computational and memory behavior, which becomes very relevant under hardware constraints.

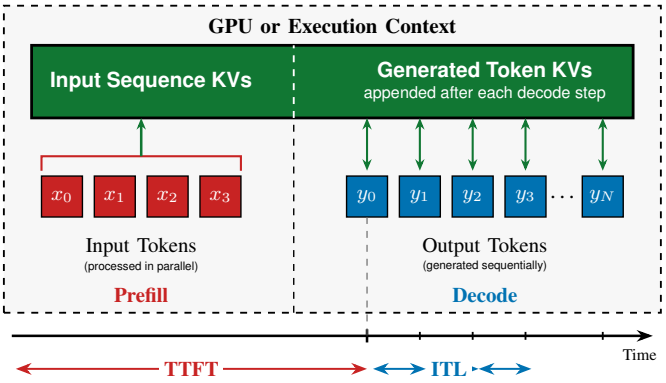


Fig. 2. Timeline of LLM inference showing the prefill and decode phases and system-level metrics. Time to first token (TTFT) measures prefill duration and any prior scheduling delay; inter-token latency (ITL) measures per-token decode step generation times.

A. LLM Inference Dynamics

Inference occurs in two operational phases with inherently distinct characteristics: prefill and decode (Fig. 2). During the **prefill** phase, the model computes key and value vectors for the entire input sequence in parallel, storing initial representations in the Key-Value (KV) cache. This stage involves high-dimensional matrix multiplications and relies heavily on the system's hardware compute capabilities.

Output generation occurs during the **decode** phase, where autoregression is used to produce tokens one step at a time. Unlike prefill, decode only processes a single token vector per step, using all previous vectors in the KV cache to avoid redundant computations. As a result, every step requires repeated reads of all model weights and the full cache for token generation. On a single GPU, decode is limited by memory bandwidth due to these frequent memory accesses. In distributed settings, decode becomes heavily exposed to communication because it has relatively little computation available to overlap collective synchronization operations compared to prefill.

The KV cache itself further impacts inference behavior by putting memory pressure on the system, scaling with sequence length, model precision, depth, hidden dimension, and batch size. Together, these prefill, decode, and KV cache dynamics result in distinct communication, computation, and memory patterns that are essential in analyzing performance under hardware constraints.

B. Inference Performance Metrics

Throughout this work, we evaluate inference performance using the following key system-level metrics [12]:

- **Time to First Token (TTFT)** – duration between request arrival and the first generated token; includes scheduling delay, tokenization, and prefill. Reflects prefill performance and scales with input length.

- **Inter-Token Latency (ITL)** – latency between successive generated tokens in decode. Reflects per-step decode performance and remains relatively constant under steady-state execution.
- **Output Throughput (TPS)** – system-wide token generation rate in tokens per second, defined as:

$$\text{TPS} = \frac{N_{\text{out}}}{T_{\text{total}}} \tag{1}$$

where N_{out} is the total number of generated output tokens across all served requests, and T_{total} is the total engine execution duration. Describes overall efficiency of the inference system and is influenced by both sequence length and concurrency.
- **Energy Cost per Token** – GPU power draw sampled at 20 Hz via `nvidia-smi`, integrated across all active GPUs, and divided by total generated tokens N_{out}, reported in Joules per token (J/tok). Enables energy efficiency comparison and aligns with other LLM energy profiling techniques in [13], [14].

Together, these metrics provide insight into inference system behavior, allowing us to observe performance changes and identify where hardware, workload, and distribution configurations matter most.

C. Distributed Infrastructure & Parallelism Methods

Different parallelization methods, hardware, and serving frameworks facilitate distributed inference and significantly affect serving system performance.

1) Parallelism Strategies: Parallelism schemes allow LLMs to run across multiple GPUs and nodes by distributing workload across available hardware [15]. **Data parallelism**, often used in training, replicates model weights and splits data across available devices [16], but can be disadvantageous for inference as it does not reduce per-GPU memory pressure. Instead, **model parallelism** schemes are favored, where a model's weights are distributed across multiple devices. Model parallelism takes two main forms.

Pipeline parallelism distributes a model by allocating sequential groups of layers across devices, with activation tensors forwarded between stages using point-to-point transfers [17]. This strategy is useful when scaling across host network interfaces (e.g., Ethernet or InfiniBand via Network Interface Cards (NICs)), which have higher latency and lower bandwidth than intra-node links, because it requires less frequent communication than its counterpart, tensor parallelism, and is therefore primarily used for inter-node parallelization [3], [18].

Alternatively, **tensor parallelism** divides individual parameter matrices of each layer across multiple GPUs, such that each device holds a partition of the model weights [19]. This method requires a collective AllReduce synchronization step for each feedforward network (FFN) and attention block to communicate all intermediate activations across GPUs, as illustrated in Fig. 3. AllReduce calls operate on tensors of shape `[batch, input length, hidden dim]` during prefill, communicating a larger volume of data across GPUs. In decode, message payloads are of shape `[batch, 1, hidden dimension]`, yielding smaller message sizes that are often more sensitive to communication latency than link bandwidth. Because AllReduce communication consumes a significant portion of iteration time in distributed workloads, tensor parallelism is usually restricted to GPUs within a single node, where high-bandwidth interconnects such as NVLink prevent significant performance degradation [5]. Throughout this paper, we focus on tensor parallelism (TP) because it represents the primary distribution strategy for intra-node LLM serving.

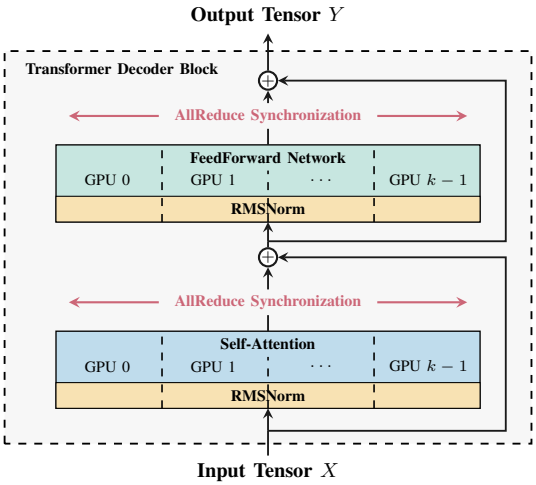


Fig. 3. Tensor parallelism architecture within a single transformer decoder block. Model weights are partitioned across k GPUs, requiring AllReduce steps following each attention and FFN block.

2) GPU Hardware and Interconnects: LLM inference typically requires GPUs, which can differ in compute capability, available memory (VRAM), and memory bandwidth, all of which directly impact inference performance. Because a single GPU may be unable to store all model parameters, activation tensors, and the KV cache required for serving, large models are often deployed across multiple devices, potentially spanning multiple nodes. Within a node, GPUs communicate via high-speed interconnects such as NVLink or PCIe. Across nodes, communication travels over network fabrics such as Ethernet or InfiniBand via host NICs, which have higher latency and lower bandwidth than intra-node links.

Inter-GPU communication is often managed by the NVIDIA Collective Communications Library (NCCL), which provides optimized collective operations such as AllReduce and point-to-point transfers. NCCL automatically selects a transport protocol based on available hardware topology and system accessibility, enabling data exchange across GPUs and nodes [20]. In Fig. 4, we illustrate an overview of GPU communication paths. Because transport protocols and physical interconnects have different latency and bandwidth characteristics, the underlying hardware topology strongly influences inference performance.

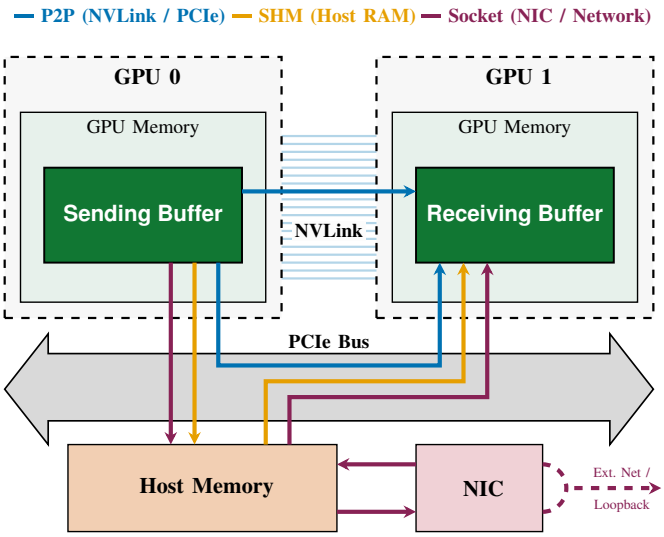


Fig. 4. High-level overview of GPU communication paths [20]. Devices within a node can communicate via interconnects such as the standard PCIe bus or direct NVLink. NCCL transports (P2P, SHM, Socket) represent the software protocol paths that move data across physical links.

D. Serving Infrastructure (vLLM)

There exist several inference serving frameworks that support efficient deployment of LLMs at scale, including TensorRT-LLM [21] and DeepSpeed Inference [18]. In this work, we use vLLM [22], an open-source inference serving framework built around PagedAttention, a KV cache optimization that treats the cache similarly to OS-paged virtual memory. The KV cache is divided into fixed-size noncontiguous blocks, all mapped to physical memory via block tables managed by vLLM's KV block manager, which allocates and releases blocks as needed during generation and request completion. This avoids the internal fragmentation, reservation waste, and external fragmentation that standard KV cache allocation creates under unknown or varying sequence lengths [22].

To further improve throughput, vLLM uses continuous batching, which adds incoming requests to ongoing batches as soon as KV slots become available, rather than waiting for a fixed batch to finish processing. The vLLM engine executes in steps, where each step evaluates available token budget and cache capacity and schedules requests using a first-come-first-served queue to process decode steps and new prefills from the waiting queue. vLLM also supports additional optimizations such as prefix caching and chunked prefill, but these are not directly relevant to this work. vLLM was chosen for its native parallelism support, well-documented open-source software, and integrated benchmarking tools.

III. Methodology

Our evaluation methodology follows standard system-level benchmarking practices to ensure reproducible results [23], [24].

A. Hardware and Models

Experiments were conducted on two servers with different GPU generations and interconnects: `shy-fec` (8× NVIDIA A100-40GB, NVLink) and `funnotch` (4× NVIDIA V100-32GB, PCIe). Table I summarizes hardware specifications [25]–[27].

TABLE I. GPU Hardware Specifications

Specification	A100 40GB	V100 32GB
Architecture	Ampere	Volta
VRAM	40 GB HBM2e	32 GB HBM2
Memory Bandwidth	1555 GB/s	900 GB/s
CUDA Compute Capability	8.0	7.0
Inference Precision	BF16	FP16
Tensor Core TFLOPS	312	130

Three of Meta's Llama models were evaluated, all dense decoder-only transformers (Table II): Llama 3.2-1B and Llama 3.1-8B use GQA [28], while Llama 2-13B uses standard MHA [29], resulting in a larger per-request KV cache footprint. Llama 3.1-8B served as the primary model for single-GPU characterization; all three were used to study the effect of model scale on distributed scaling behavior.

TABLE II. Parameters of Evaluated Llama Model Architectures

Specification:	Llama –	3.2-1B	3.1-8B	2-13B
Transformer Layers		16	32	40
Model Dimension		2048	4096	5120
Attention Heads		32	32	40
KV Heads		8	8	40
Attention Type		GQA	GQA	MHA
Weight Footprint (16-bit)		∼2GB	∼16GB	∼26GB

All experiments used vLLM 0.11.2 with GPU memory utilization fixed at 0.85. Twenty warmup requests were sent before measurement to allow GPUs to reach steady state, and the end-of-sequence token was ignored to allow full control of output lengths. For sequence length control and reproducibility, all experiments used vLLM's synthetic RandomDataset; prior work [23] found minimal differences in performance between synthetic and real inputs, confirming that synthetic datasets can approximate real workload characteristics.

B. Single-GPU Characterization Setup

Single-GPU experiments established a baseline before introducing inter-GPU communication, using two workload sweeps to isolate model and hardware behavior from communication effects. We swept across input and output sequence lengths (ISL/OSL), each spanning {128, 256, 512, 1024, 2048} tokens, with concurrency fixed at 1 to isolate model behavior without batching effects. This range captures various use cases such as generation (ISL $<$ OSL), translation (ISL $\approx$ OSL), and summarization (ISL $>$ OSL) [30]. Each configuration was repeated across 3 runs with 100 prompts per run. A concurrency sweep was conducted to examine how the number of simultaneous requests affects system behavior, sweeping up to 512 concurrent requests. Each configuration was repeated

across 2 runs and used 1000 prompts to ensure stable metrics. The inference system demonstrated strong run-to-run stability, with maximum relative standard deviation 1.6% for TPS and mean ITL, and mean TTFT remaining below 1% for most configurations, but peaking at 21% for highest concurrency levels, where request queuing increases timing differences between runs.

C. Multi-GPU Setup: Tensor Parallelism and Transports

Distributed experiments used tensor parallelism degrees TP $\in$ 1,2,4 with ISL/OSL fixed at 512 to approximate a conversational inference workload. Concurrency was varied up to 2048 to observe behavior below and above KV cache saturation. A sufficient number of prompts were processed to ensure metrics were captured under sustained load. Given the low run-to-run variance found in single-GPU characterization and resource constraints, most multi-GPU configurations were evaluated with single runs.

Following a standard tensor-parallel scheme [19] shown in Fig. 3, attention and FFN projections are split across GPUs, resulting in two AllReduce operations per transformer layer (32, 64, and 80 total per forward pass for the 1B, 8B, and 13B models, respectively). To isolate the sensitivity of distributed inference to communication quality, three NCCL transport configurations were evaluated by controlling environment variables, forcing hardware fallback behavior while keeping the model and workload fixed. Disabling peer-to-peer paths effectively switches traffic off of NVLink to PCIe paths when both interconnects exist [31].

- **Default (GPUDirect P2P):** direct GPU-to-GPU memory access over NVLink or PCIe peer-to-peer (P2P), bypassing host memory and the network stack.
- **CPU Shared Memory (SHM):** enforced with `NCCL_P2P_DISABLE=1`; disables P2P and routes tensors through host system CPU memory, introducing host PCIe bus and memory copy overhead.
- **TCP Socket:** enforced with `NCCL_P2P_DISABLE=1` and `NCCL_SHM_DISABLE=1`; disables both P2P and SHM, forcing inter-GPU communication through host CPU staging buffers and the operating system's TCP/IP network stack (e.g., over loopback or Ethernet).

To ensure that transport was selected as expected, NCCL debug logs were monitored, and vLLM's custom AllReduce kernels were disabled, forcing all inter-GPU synchronization to route through vLLM's PyNCCL communicator to ensure the use of NCCL collectives. Measured AllReduce latency across these tiers (seen in Fig. 5), obtained via `nccl-tests` [32], confirms that NVLink P2P consistently achieves the lowest operational latency, with SHM and socket transport introducing order-of-magnitude higher communication latency. Larger message sizes also increase operational latency across all transports.

D. Latency Injection Mechanism

To study how communication latency affects distributed inference independent of the transport, where bandwidth and protocols differ, artificial latency was injected into the AllReduce call path within vLLM's PyNCCL communicator, immediately preceding its call to `ncclAllReduce`. A GPU sleep (`torch.cuda._sleep()`) delays the communication stream by forcing the GPU to spin for a target number of GPU clock cycles, computed as:

$$\text{cycles} = \Delta t_{\mu s} \times f_{\text{clock, MHz}} \tag{2}$$

using the A100's active graphics clock frequency (1410 MHz), obtained via `nvidia-smi`. These sleeps are captured by CUDA graphs during warmup and replayed for all subsequent decode steps. This injection approach was chosen because standard network emulation tools (e.g., Linux's `netem`) operate on the network stack and cannot intercept intra-node P2P or SHM traffic. Modifying physical links is not user-programmable; thus, the NCCL AllReduce call is left as the lowest viable level at which communication can be intercepted and modified. This method provides an idealized, pure delay and does not capture other link degradation characteristics such as jitter or packet loss.

Injected latency values (0, 100, 500, 1000, 1500, 2000, 2500 μs) were chosen to capture the gap between best-case NVLink and worst-case SHM/socket AllReduce latency, as characterized by Fig. 5. To validate the injection mechanism, the measured ITL change was compared against the predicted change for a decode step (injected latency $\times$ AllReduce operations per forward pass). Injection experiments were performed on the A100 machine for two concurrency levels (32 and 512), representing pre- and post-KV cache saturation workloads.

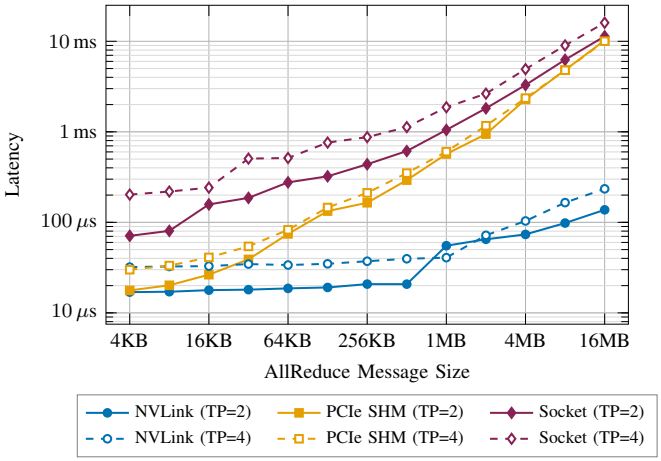


Fig. 5. Microbenchmark latency of NCCL AllReduce operations across tensor payload sizes for TP=2 and TP=4 on the A100 server for P2P, SHM, and socket transports.

IV. Inference Performance and Scaling Characterization

A. Single-GPU Baseline

We first characterize single-GPU inference behavior across sequence lengths and request concurrency on A100 and V100 GPUs, isolating model and hardware behavior before introducing inter-GPU communication.

1) Impact of Sequence Length: Across sequence length configurations of up to 2048 tokens each, TTFT scaled primarily with input length, rising from 24 to 144 ms on the A100 and 62 to 782 ms on the V100, while varying at most by 2 ms across OSLs for any fixed ISL. The A100 GPU maintained faster TTFT compared to V100 (an average of 64 ms vs. 270 ms across all configurations) due to its better compute capabilities.

On the other hand, decode performance stayed stable across sequence length: mean ITL ranges from only 13.25–13.62 ms on the A100 (0.36 ms spread) and 24.2–27.5 ms on the V100 (3.26 ms spread). This stability reveals the main distinction between prefill and decode: prefill processes an entire input sequence in parallel, while decode only generates one new token per step, so execution time is mostly determined by GPU memory bandwidth rather than sequence length. The observed ITL ratio between GPUs, using the average ITL across the sweep ($25.9 / 13.4 \approx 1.9\times$), lines up closely with their memory bandwidth ratio ($1555 / 900 \approx 1.7\times$), with discrepancies likely due to other A100 advantages. Output throughput followed the same stability pattern, ranging from 68.9–75.2 tokens/s on the A100 and 30.9–40.9 on the V100, reaching its lowest with high ISL / low OSL configurations where more time is spent in prefill rather than generation.

2) Concurrency and KV Cache Saturation: As request concurrency increases, vLLM manages more simultaneous sequences in the KV cache, which increases memory usage and significantly impacts behavior once saturated. The KV cache size is determined by vLLM at server instantiation using the formula:

$$\begin{aligned}\text{KV cache} = \text{VRAM} \times \text{gpu_mem_util} \\ - (\text{weights} + \text{activations} + \text{overhead})\end{aligned} \tag{3}$$

and required memory per token given by [33]:

$$\text{KV bytes per token} = 2 \times n_{\text{layers}} \times n_{\text{KV heads}} \times d_{\text{head}} \times d_{\text{dtype}} \tag{4}$$

$$\text{Total Sequence Cache} = S \times \text{KV Bytes per Token} \tag{5}$$

where S is the number of tokens in the cache at a given point in time, which we estimate assuming mid-decode (ISL + OSL / 2). Peak activations during computation and framework overhead vary depending on workload characteristics, but in practice amount to approximately 5–10% of GPU memory [34]. As a result, we find that for Llama 3.1-8B, this predicts a saturation point of approximately 150 concurrent requests on the A100 (14–16 GB KV cache) and 90 on the V100 (8–10 GB KV cache).

Output throughput and mean ITL are shown against concurrency in Fig. 6. Both rise as the number of concurrent requests increases, until the KV cache approaches saturation and both metrics plateau. At low concurrency, the GPU only processes a few sequences per step, leaving GPU resources underutilized. As concurrency increases, more sequences are batched together in each forward pass, increasing output throughput and ITL, because every decode step now requires access to a larger number of cache blocks. Both metrics

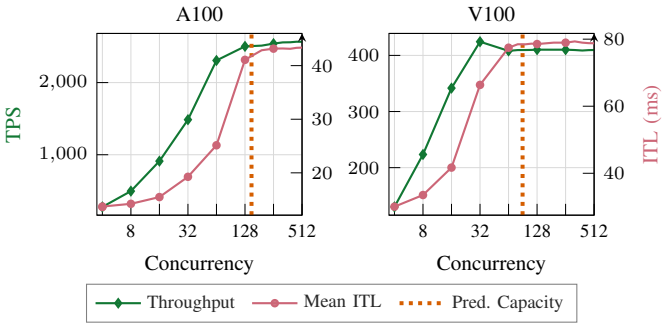


Fig. 6. Output throughput (left axis, blue) and mean ITL (right axis, red) of Llama 3.1-8B as concurrency scales up to 512 requests on both GPUs. Dashed vertical lines mark the analytically predicted KV cache saturation point.

plateau, aligning well with the estimated saturation point, as the KV cache reaches saturation and the amount of processed data per step stops increasing. TTFT behaves differently under concurrency, gradually increasing pre-saturation due to growing batch sizes, then beginning to climb rapidly without a ceiling post-saturation, where significant request queuing delay slows down TTFT.

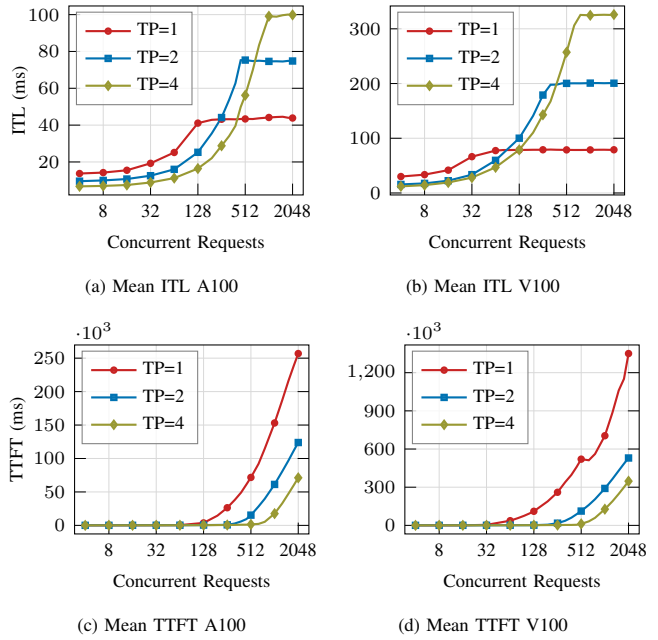
In practice, served workloads have varying ISL/OSL distributions, shifting the saturation point depending on the current workload. This controlled, fixed-sequence experiment provides insight into how the saturation point influences performance metrics. In multi-GPU settings, available cache memory increases, pushing the saturation point higher, while slower inter-GPU communication lengthens decode steps and determines how performance degrades.

B. Scaling with Tensor Parallelism

Tensor parallelism provides additional compute and memory, along with the cost of communication collectives required to synchronize partial activations across devices. Because prefill and decode have inherently different exposure to communication collectives, they respond differently to scaling.

1) Decode Performance: When operating below KV cache capacity, increasing TP degree improves ITL by distributing model weights, KV cache tensors, and matrix multiplications across GPUs. At light workloads, the reduced memory-access latency and computational speedup of decode steps outweigh the cost of tensor synchronization. As concurrency approaches and exceeds KV cache saturation, ITL curves reach a ceiling that is higher for higher TP degrees, because larger systems can sustain more concurrent requests. Each decode step then operates on larger batch sizes, increasing both per-step computation and the size of AllReduce tensors exchanged between GPUs, raising the steady-state decode latency.

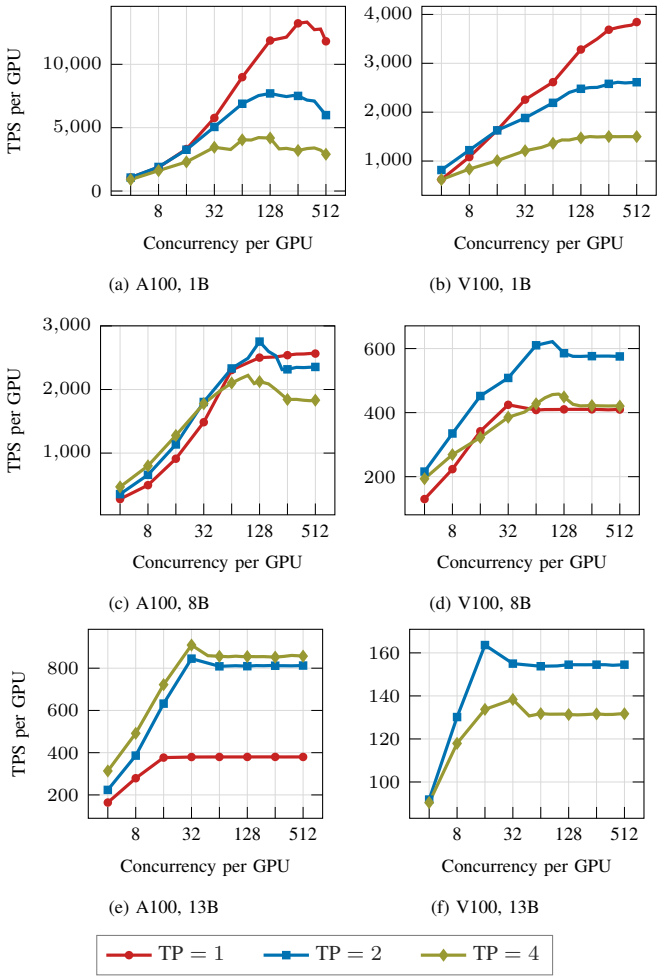
The crossover point, where a higher TP degree stops reducing and starts increasing ITL, marks the workload where the communication overhead of larger AllReduce operations exceeds the computational benefit of parallelization. Both GPU servers exhibit this behavior, although the V100 consistently shows higher overall ITL due to its lower computational



(a) Mean ITL A100 (b) Mean ITL V100

(c) Mean TTFT A100 (d) Mean TTFT V100

Fig. 7. Average ITL (a, b) and TTFT (c, d) of Llama 3.1-8B across concurrency levels for TP = 1, 2, and 4 on A100 (NVLink) and V100 (PCIe) GPUs.



(a) A100, 1B (b) V100, 1B

(c) A100, 8B (d) V100, 8B

(e) A100, 13B (f) V100, 13B

Fig. 8. Per-GPU output throughput (tok/s) scaling for Llama 3.2-1B, Llama 3.1-8B, and Llama 2-13B models across TP = 1, 2, and 4 devices on A100 (left) and V100 (right) servers. Both throughput and concurrency are normalized by the TP degree.

throughput and slower PCIe interconnect compared to the NVLink-connected A100 server.

2) Prefill Performance: Below KV cache capacity, TTFT maintains relatively low latency across TP degrees (Fig. 7c and 7d), as the additional AllReduce communication introduced can be overlapped by prefill's compute-heavy matrix multiplications. However, once concurrency approaches KV cache saturation, TTFT quickly rises, driven by larger message sizes and request queuing delays as incoming requests must wait for KV cache blocks to become available before prefill can begin. Because the V100 server has less available VRAM per GPU, and therefore lower KV cache capacity, saturation occurs at a lower concurrency, causing TTFT to degrade earlier.

These results tie the two inference phases together to a common saturation boundary. Below capacity, TP benefits both phases by parallelizing model computations. Beyond saturation, both phases exhibit degraded performance due to prefill's queuing delay and decode's larger AllReduce message payloads and increased per-step computation.

3) Scaling Efficiency: Moving on from latency to throughput, we normalize both output throughput and concurrency by TP degree to compare per-GPU efficiency across configurations. This normalization accounts for the scaling of compute and memory resources to evaluate whether additional hardware is used effectively. Fig. 8 presents this comparison across evaluated models and hardware platforms.

Continuing with Llama 3.1-8B (Fig. 8, middle row) on the NVLink-connected A100 server, TP $= 2$ slightly exceeds the single-GPU baseline at low concurrency, where the added compute and memory still outweigh communication cost, then falls below baseline as concurrency nears saturation. TP $= 4$ shows an earlier baseline crossover and lower normalized throughput than TP $= 2$, suggesting that additional synchronization costs incurred by further distributing from 2 to 4 devices tip the tradeoff toward diminishing returns. The scaling behavior is different on the PCIe-connected V100 server, where scaling to two GPUs (TP $= 2$) provides a clear increase in per-GPU throughput over the entire sweep, while TP $= 4$ and TP $= 1$ track each other closely. This suggests that distributing the 8B model across two GPUs on the V100 machine offers the most efficient hardware utilization, as the V100's limited VRAM gains a greater advantage from the additional KV cache capacity and higher batch sizes than on the A100, though scaling further to four GPUs does not extend the benefit.

The remaining rows of Fig. 8 show a smaller and larger model evaluated across both setups to understand whether this pattern extends beyond Llama 3.1-8B. The results demonstrate that the resource efficiency of the serving system depends strongly on model characteristics as well as hardware. Communication overhead under TP scales with model depth, as

more transformer layers mean more per-decode step AllReduce operations. The weight footprint of a model also increases as parameter count increases, leaving less VRAM for the KV cache. Attention mechanisms can also impact per-request KV cache cost: MHA stores separate KV projections for each attention head, while GQA shares KV heads across multiple query heads, reducing the KV cache footprint. As a result, Llama 2-13B's MHA mechanisms lead to much larger cache footprints per token than the GQA-based Llama models. The benefit of distributing across additional GPUs depends on how much KV cache pressure a model puts on the system, and how much additional memory capacity improves achievable request concurrency.

This trade-off results in different behavior across the three models. Llama 3.2-1B fits comfortably on a single GPU (TP $= 1$) and gains little from additional memory capacity of distribution. Conversely, the larger Llama 2-13B model has both a higher communication frequency and a larger memory footprint of approximately 26 GB, preventing deployment on a single V100 where available VRAM is insufficient. This memory pressure results in Llama 2-13B benefiting more from additional GPUs, despite its added AllReduce overhead, though the optimal TP configuration still depends on the underlying hardware. On the A100 NVLink server, the 4-GPU configuration is most efficient, while on the V100 PCIe server, despite being more memory constrained, TP $= 2$ remains the optimal configuration, suggesting that interconnect quality plays a role independent of memory pressure.

Overall, TP scaling efficiency is determined by the balance between synchronization overhead and the memory/compute capacity gained from additional GPUs, making the optimal configuration both model- and hardware-dependent. However, pure TP scaling masks true interconnect sensitivity because the added resources can overlap with communication stalls while NCCL defaults to optimal transport paths. The next section isolates communication overhead from these resource gains by controlling transport tiers, injecting collective communication latency, and evaluating their direct impact on performance latency and resource efficiency.

V. Effect of Communication on Performance

A. Transport Sensitivity

Thus far, NCCL has been free to automatically select the best available transport. Here, we constrain communication onto progressively slower transport tiers to isolate interconnect quality as a variable, and observe how the communication layer alone shapes inference performance.

1) Impact on Inference Latency: Inference latency metrics follow the same degradation patterns previously established in Sec. IV. TTFT stays relatively flat pre-saturation and climbs rapidly without a ceiling post-saturation as queuing delays begin to accumulate. Over lower-tier transports, TTFT quickly degrades to levels unacceptable for production serving, while high-performing interconnects enable speedy prefills that overcome the compute-communication tradeoff. ITL curves separate quickly based on the underlying transport path: direct

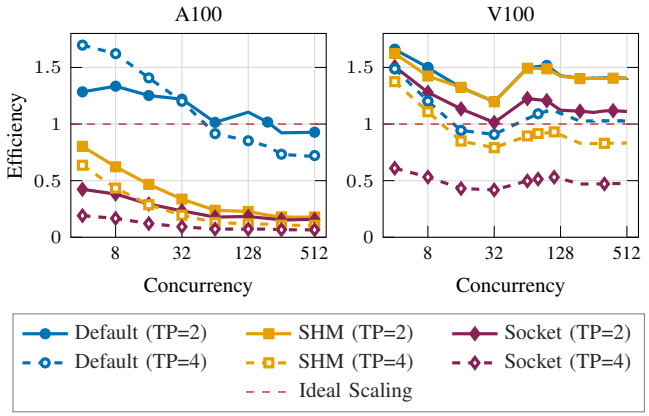


Fig. 9. Throughput scaling efficiency ($\text{TPS}_{TP=k}/(k \cdot \text{TPS}_{TP=1})$) versus concurrency of Llama 3.1-8B on A100 and V100 systems across transport tiers.

NVLink P2P keeps lower ITL ceilings well below 200 ms per decode step, whereas forcing communication through SHM or socket paths raises the post-saturation ceiling significantly. Higher TP degrees amplify this gap further due to additional link traversals required during AllReduce operations.

Combining the measured ITL with the `nccl-tests` microbenchmarks (Fig. 5) and the calculated decode message size (`[batch, 1, hidden dimension]`) provides an estimate of the total execution time spent on collective communication per decode step. On the A100 machine, NVLink maintained the lowest AllReduce overhead, only 6–14% of ITL for TP $= 2$ and 10–33% for TP $= 4$ across concurrency levels. Without P2P, the overhead increases significantly, accounting for 21–38% and 37–48% (TP $= 2, 4$) using SHM, and even larger overheads using TCP/IP sockets of 41–59% and 47–78% (TP $= 2, 4$). As shown in Sec. V-C, estimates derived from microbenchmarks likely underestimate the true communication overhead, yielding an optimistic lower bound on communication cost.

2) Impact on Scaling Efficiency: To evaluate how effectively tensor parallelism scales across transport layers, we compare throughput scaling efficiency relative to the single-GPU baseline, where values below 1 indicate sub-linear scaling (Fig. 9).

On the A100 platform, where NCCL defaults to GPUDirect P2P over NVLink, scaling efficiency remains above 1 until high concurrency levels. However, when directing communication over SHM or socket paths, scaling efficiency drops significantly across all TP degrees.

The V100 system demonstrates how physical hardware topology shapes scaling efficiency. Because its asymmetric topology leaves the TP $= 2$ GPU pair without direct P2P support, NCCL's default execution already relies on shared memory, making forced SHM performance identical. For TP $= 4$, a gap forms between default and forced SHM because the default NCCL path can still use the P2P-enabled GPU pair within its AllReduce ring, speeding up communication. Additionally, on memory-constrained hardware like the V100

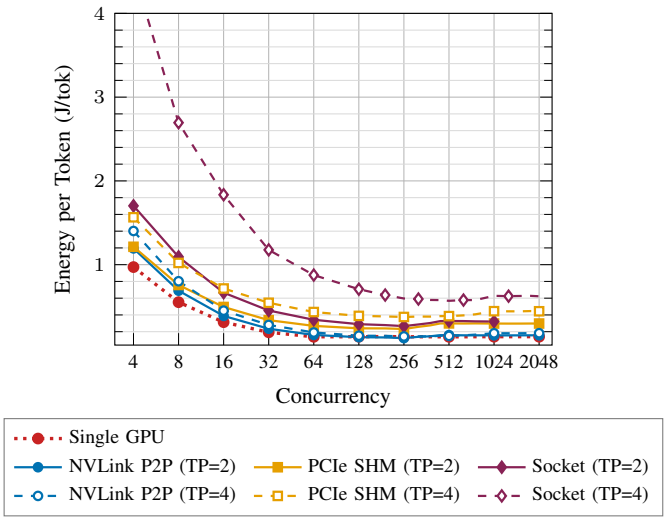


Fig. 10. Energy consumption in Joules per token of Llama 3.1-8B across concurrency levels and TP degrees when routing communication via evaluated NCCL transports (P2P over NVLink, CPU Shared Memory using PCIe bus, and TCP Socket)

machine, the extra KV cache capacity helps offset the added communication costs, maintaining higher relative scaling efficiency than on the less constrained A100 platform.

3) Impact on Energy Consumption: Energy consumption per token (measured in Joules) also provides insight into resource efficiency across transports, and is closely tied to request concurrency (Fig. 10). Across all transport tiers, energy cost per token decreases with increasing concurrency before leveling off at high batch sizes as GPU compute utilization saturates. However, as transport quality degrades, curves plateau at higher baseline energy costs.

Transport protocols across interconnects with lower bandwidth and higher latency cause GPUs to spend more time idling during decode steps while waiting on partial activation tensors from other devices. Although GPU instantaneous power draw drops during these synchronization stalls, the significant increase in total execution time results in a higher cumulative energy consumption during inference. Compared to the default NCCL P2P transport selection, forcing SHM transport increases energy consumption per token by up to 90% for TP $= 2$ and 165% for TP $= 4$. The impact is more severe over socket connections, where energy cost increases up to 115% for TP $= 2$ and 365% for TP $= 4$.

Ultimately, these findings show that both TP degree and transport quality determine whether distributing a model is worth the communication cost.

B. Latency Injection

To isolate communication latency itself, independent of the protocol overheads and bandwidth differences that separate host SHM, TCP sockets, and NVLink P2P, we investigate how performance changes in response to latency added onto AllReduce operations.

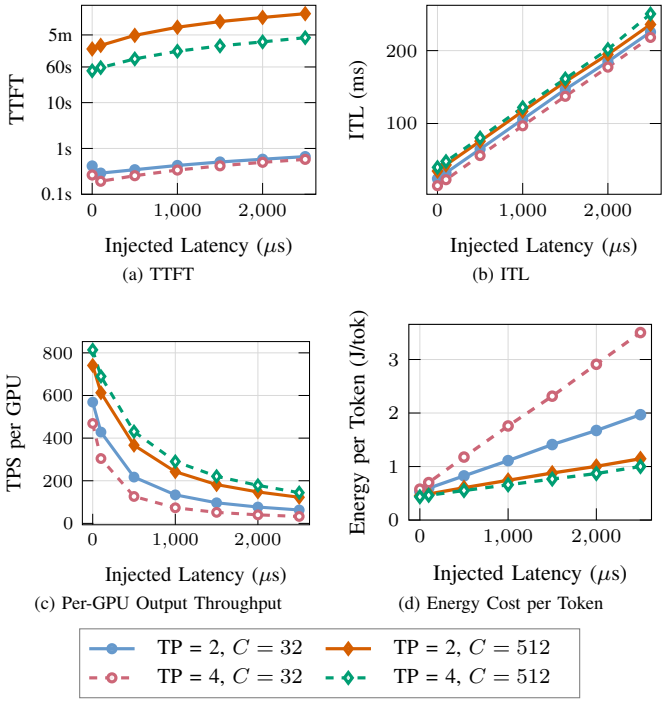


Fig. 11. Inference system response to AllReduce latency injection for Llama-2-13B on A100 GPUs at light and post-saturation loads ($C = 32, 512$).

1) Mechanism Validation: Before interpreting the impact of added communication latency on performance, we validate that the injection mechanism changes ITL as expected. Distributing a transformer model of L layers using tensor parallelism yields two AllReduce operations per transformer layer, one each following the attention block and the FFN block. One decode step, as measured by ITL, completes a single forward pass featuring $2L$ AllReduce operations.

Injecting an AllReduce delay ΔAR into every AllReduce call therefore must result in an ITL increase of:

$$\Delta\text{ITL} = 2L \times \Delta\text{AR} \tag{6}$$

For Llama-2-13B ($L = 40$), one decode step executes 80 AllReduce calls, resulting in the expected increase of 80 ms per injected millisecond of latency. Fig. 11b demonstrates this linear scaling, with empirical measurements matching the predicted 80 ms slope within a 5% error margin across measurements.

2) Impact on Inference Latency: ITL increases linearly with added AllReduce communication latency across all setups. The TP or concurrency configuration determines the offset between the curves, but ITL degrades at the same rate with respect to added communication latency regardless of workload because the number of communication calls remains the same.

TTFT, unlike ITL, has a different rate of degradation depending on concurrency (Fig. 11a). High concurrency workloads ($C = 512$) see much higher TTFTs and are more sensitive to injected latency. Low concurrency workloads ($C = 32$) yield flatter curves, though still increase linearly under added communication latency. The difference can likely be attributed

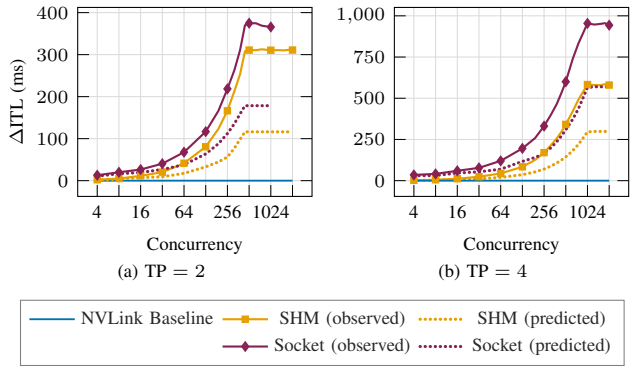


Fig. 12. Predicted and observed ΔITL of Llama 3.1-8B for SHM and socket transports relative to default NVLink P2P baseline. Predicted values obtained using the AllReduce delay model on `nccl-tests` microbenchmark latencies.

to the queuing that occurs at higher concurrency, where slower inference steps delay scheduling of incoming requests.

3) Impact on Throughput and Energy Cost: Per-GPU output throughput decreases hyperbolically in response to injected latency (Fig. 11c), reflecting the inverse relationship between throughput and decode step latency [12]. Low concurrency workloads ($C = 32$) see a steeper relative throughput drop because communication accounts for a larger portion of total execution time per step.

Energy cost per generated token increases linearly with injected AllReduce latency (Fig. 11d). High concurrency configurations ($C = 512$) remain more energy efficient because more concurrent requests can share the cost of communication. The slopes of the curves reveal the system's sensitivity to communication latency, showing that low concurrency configurations are the most vulnerable. The significant increase in total execution time due to latency-caused synchronization stalls results in higher cumulative energy consumption during inference, despite lower instantaneous power draw.

C. Applying the Latency Model to Transport Fallback

Using the idealized AllReduce latency model validated by controlled latency injection (6), we test whether collective communication latency alone can predict the performance degradation observed when falling back to lower-tier transports. To see if the model generalizes beyond controlled delay, we apply (6) to the measured AllReduce latencies in Fig. 5 to predict decode-latency degradation (ΔITL). Because AllReduce message size depends on the batch size processed within a decode step, we cap the effective batch at the estimated KV cache capacity calculated using the same methods described in Sec. IV-A2.

Fig. 12 compares predicted and observed values of ΔITL, defined as the difference in ITL between default and fallback transports for matching configurations. The injection model accurately predicts the overall linear trend, but it consistently underestimates the actual decode-step degradation. Across all tested concurrencies and transports for the 8B model, the underestimation error ranges from 14–66%, increasing with concurrency.

This underestimation reveals that the pure latency model captures only part of the overhead incurred by falling back to sub-optimal transports. The `nccl-tests` microbenchmarks isolate communication collectives from the systems that use them, whereas real serving environments often involve significant concurrent activity, such as request scheduling and memory allocations. These processes, along with other transport-specific overheads, such as host-memory staging, may contribute to the additional decode step latency we observe here. These results indicate that our latency injection findings represent an optimistic lower bound on the real cost of lower-tier transports.

D. Service Level Objective Evaluation

Lastly, we briefly examine Service Level Objectives (SLOs) that often constrain deployed LLMs in practice to ensure satisfactory service for users. Following responsive SLO frameworks [35], we define targets based on the 90th percentile of metrics, holding TTFT to under 1 second and ITL under 100 ms. These provide quick insights on serving distributed LLMs and help identify which workloads allow higher capacities or tolerate higher communication latency before performance degrades.

Fig. 13 shows that smaller model sizes leave room for more concurrent requests within SLO attainment, while larger model sizes have lower capacities before failing to reach constraints, with the older V100 GPU generation maintaining lower SLO-capacity.

Latency injection evaluations under light load ($C = 32$) show that smaller models can handle higher communication latency before failing to meet responsive SLO targets, unlike high concurrency ($C = 512$) workloads, where using default NCCL paths violates SLOs without added latency. Llama 3.2-1B (TP $= 2$) handles up to 2450 μs of injected AllReduce latency, compared to 1340 μs and 1140 μs for Llama 3.1-8B (TP $= 2, 4$), and 1020 μs and 900 μs for Llama 2-13B (TP $= 2, 4$). This framing connects evaluated configurations and presents operational limits for balancing workloads, deployment scales, and hardware constraints in LLM serving.

VI. Conclusions

This paper investigates how communication infrastructure impacts intra-node distributed language model serving using tensor parallelism. Specifically, we study transport protocols and collective communication latency and their impact on whether a model benefits from further distribution. The resulting findings lead to the same conclusion: scaling decisions must be made hardware-aware, not based on compute and memory budgets alone.

System factors like model depth, request volume, Key-Value (KV) cache saturation, and physical hardware topology—including both accelerators and interconnects—all shape a model's performance and scaling efficiency. Because tensor parallelism inference requires frequent AllReduce operations

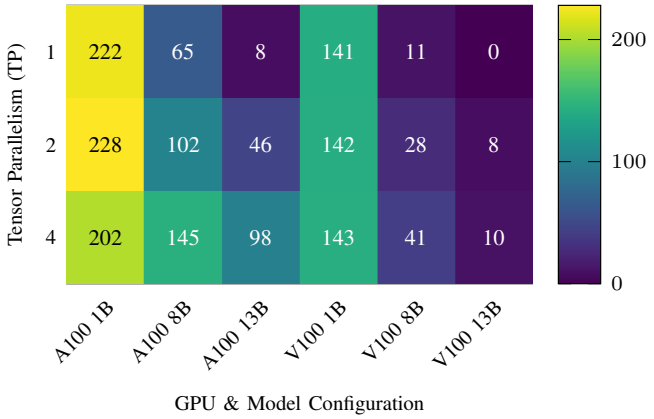


Fig. 13. Heatmap displaying maximum concurrent request capacity while satisfying SLOs (TTFT $\leq$ 1 s and ITL $\leq$ 100 ms at the 90th percentile) across model sizes, GPU generation, and TP degrees.

for GPU synchronization, latency from recurring communication directly impacts performance by slowing down forward passes and causing stalls that significantly increase energy cost due to GPU idling. As concurrency increases, KV cache saturation marks a key point in performance profiles where request queuing delays begin to compound and degrade performance further than communication overhead alone would predict. Transport protocol directly determines whether model distribution is advantageous, along with model size, KV cache pressure, and concurrency, which all interact and shift the optimal configuration depending on workload.

These insights are relevant past the specific hardware studied here. As models continue to grow and GPUs continue to improve in memory and compute capabilities, the relevance of interconnects may become even more pronounced as compute increasingly outpaces communication. While this work focuses on dense models up to 13 billion parameters on modest setups, the findings remain applicable to cost-constrained, limited-hardware serving environments. Extending this methodology to inter-node topologies, where latency and bandwidth constraints are more severe and collective communication patterns differ, is a natural next step to providing broader insights into distributed serving. Future work could also extend our findings by evaluating other workload patterns, non-dense architectures like Mixture of Experts models, and modern serving optimizations like weight quantization and prefill-decode disaggregation to see how they change communication overhead sensitivity. Frameworks could eventually use these performance profiles to dynamically adjust deployment strategies based on real-time interconnect conditions. Overall, as models continue to scale, the underlying communication layer, not just raw compute and memory, will define operational limits, energy footprints, and scalability of distributed inference systems.

Acknowledgment

The author would like to thank Prof. A. Alvarado for the opportunity to pursue this master's graduation project and for the insightful guidance throughout the research. Many thanks to the ICTLab at TU/e for providing the resources that made this work possible.

References

[1] B. Hanindhito, B. Patel, and L. K. John, "Bandwidth characterization of deepspeed on distributed large language model training," in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Indianapolis, IN, USA: IEEE, May 2024.

[2] Z. Wan, X. Wang, C. Liu, S. Alam, Y. Zheng, J. Liu, Z. Qu, S. Yan, Y. Zhu, Q. Zhang *et al.*, "Efficient large language models: A survey," *arXiv preprint arXiv:2312.03863*, May 2024.

[3] J. Kundu, W. Guo, A. BanaGozar, U. De Alwis, S. Sengupta, P. Gupta, and A. Mallik, "Performance modeling and workload analysis of distributed large language model training and inference," in *2024 IEEE International Symposium on Workload Characterization (IISWC)*. Vancouver, Canada: IEEE, Sep. 2024.

[4] Epoch AI, "Data on AI models," https://epoch.ai/data/ai-models, 2026.

[5] H. Qi, L. Dai, W. Chen, Z. Jia, and X. Lu, "Performance characterization of large language models on high-speed interconnects," in *2023 IEEE Symposium on High-Performance Interconnects (HOTI)*. Virtual Conference: IEEE, Aug. 2023.

[6] Q. Anthony, B. Michalowicz, J. Hatef, L. Xu, M. Abduljabbai, A. Shafi, H. Subramoni, and D. K. Panda, "Demystifying the communication characteristics for distributed transformer models," in *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*. Virtual Conference: IEEE, Aug. 2024.

[7] S. Cheng, J.-L. Lin, M. Emani, S. Raskar, S. Foreman, Z. Xie, V. Vishwanath, and M. T. Kandemir, "Thorough characterization and analysis of large transformer model training at-scale," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 8, no. 1, Feb. 2024.

[8] G. Mi, X. He, and X. Zheng, "Relevance of latency in ethernet networking for AI infrastructure," in *ECOC 2024; 50th European Conference on Optical Communication*. Frankfurt, Germany: VDE, Sep. 2024.

[9] L. Xu, K. K. Suresh, Q. Anthony, N. Alnaasan, and D. K. Panda, "Characterizing communication patterns in distributed large language model inference," in *2025 IEEE Symposium on High-Performance Interconnects (HOTI)*. Virtual Conference: IEEE, Aug. 2025.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, Dec. 2017.

[11] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, Dec. 2023.

[12] S. Sagi, "Optimizing LLM inference: Metrics that matter for real time applications," *Journal of Artificial Intelligence & Cloud Computing*, vol. 4, no. 1, pp. 1–4, Jan. 2025.

[13] M. F. Argerich and M. Patiño-Martínez, "Measuring and improving the energy efficiency of large language models inference," *IEEE Access*, vol. 12, pp. 80 194–80 207, Jun. 2024.

[14] J.-W. Chung, R. Wu, J. J. Ma, and M. Chowdhury, "Where do the joules go? Diagnosing inference energy consumption," *arXiv preprint arXiv:2601.22076*, Jan. 2026.

[15] J. Duan, S. Zhang, Z. Wang, L. Jiang, W. Qu, Q. Hu, G. Wang, Q. Weng, H. Yan, X. Zhang *et al.*, "Efficient training of large language models on distributed infrastructures: A survey," *Vicinagearth*, vol. 3, no. 1, p. 9, Jun. 2026.

[16] J. Fernandez, L. Wehrstedt, L. Shamis, M. Elhoushi, K. Saladi, Y. Bisk, E. Strubell, and J. Kahn, "Hardware scaling trends and diminishing returns in large-scale distributed training," *arXiv preprint arXiv:2411.13055*, Apr. 2025.

[17] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," *Advances in Neural Information Processing Systems*, vol. 32, Dec. 2019.

[18] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, "DeepSpeed-inference: Enabling efficient inference of transformer models at unprecedented

scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. Dallas, TX, USA: IEEE, Nov. 2022.

[19] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, Mar. 2020.

[20] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, "Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms," in *2025 IEEE Symposium on High-Performance Interconnects (HOTI)*. Virtual Conference: IEEE, Aug. 2025.

[21] NVIDIA Corporation, "TensorRT-LLM," https://developer.nvidia.com/tensorrt-llm, 2023.

[22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz, Germany: Association for Computing Machinery, Oct. 2023.

[23] S. Salaria, Z. Liu, and N. M. Gonzalez, "Meta-metrics and best practices for system-level inference performance benchmarking," *arXiv preprint arXiv:2508.10251*, Aug. 2025.

[24] A. Agrawal, N. Kedia, A. Agarwal, J. Mohan, N. Kwatra, S. Kundu, R. Ramjee, and A. Tumanov, "On evaluating performance of LLM inference serving systems," *arXiv preprint arXiv:2507.09019*, Jul. 2025.

[25] NVIDIA Corporation, "NVIDIA A100 tensor core GPU datasheet," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf, 2024.

[26] ——, "NVIDIA Tesla V100 tensor core GPU datasheet," https://images.nvidia.com/content/technologies/volta/pdf/volta-v100-datasheet-update-us-1165301-r5.pdf, 2017.

[27] ——, "CUDA GPU compute capability," https://developer.nvidia.com/cuda/gpus, 2026.

[28] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, Nov. 2024.

[29] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, Jul. 2023.

[30] NVIDIA Corporation, "NVIDIA NIM LLMs benchmarking documentation," https://docs.nvidia.com/nim/benchmarking/llm/latest/index.html, 2024.

[31] Y. H. Temuçin, A. Sojoodi, P. Alizadeh, and A. Afsahi, "Efficient multi-path NVLink/PCIe-aware UCX based collective communication for deep learning," in *2021 IEEE Symposium on High-Performance Interconnects (HOTI)*. Virtual Conference: IEEE, Aug. 2021.

[32] NVIDIA Corporation, "Performance reported by NCCL tests," https://github.com/NVIDIA/nccl-tests/blob/master/doc/PERFORMANCE.md, 2024.

[33] S. Verma and N. Vaidya, "Mastering LLM techniques: Inference optimization," https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/, NVIDIA Technical Blog, 2023.

[34] vLLM Team, "Gpu memory utilization," https://docs.vllm.ai/projects/vllm-omni/en/latest/configuration/gpu_memory_utilization/, vLLM Documentation, 2026.

[35] K. Hong, X. Li, L. Chen, Q. Mao, G. Dai, X. Ning, S. Yan, Y. Liang, and Y. Wang, "SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling," in *Eighth Conference on Machine Learning and Systems*, vol. 7. Santa Clara, CA, USA: MLSys, May 2025.